Technical Report

Assignment 2 — Vector Embedding for Capability Composition

Structured Capability Embedding (SCE)

Executive Summary

This report presents **Structured Capability Embedding (SCE)**, a problem-specific vector representation designed for the formal capability model in Assignment 2. The central design decision is to preserve functional structure—inputs, outputs, preconditions, effects, resources, constraints, and operational attributes—rather than relying on a generic text embedding.

Key idea: similarity answers “how functionally alike are these capabilities?”, while compatibility answers “can the first capability feed the second?” This distinction is essential because composition is directional.

1. Problem Definition

The assignment asks whether formally specified states, goals, and executable capabilities can be represented in a vector space such that the relationships required for capability compatibility, composition, and construction of complex functionality are preserved.

The formal application is $A=(S,C,S_I,G,R,K)$. Assignment 1 provides planning context, but the planner itself is explicitly outside the scope of this assignment.

2. Design Requirements

• Capability identity — functionally different capabilities must remain distinguishable.

• State awareness — the representation must relate capabilities to the states on which they operate.

• Precondition–effect compatibility — effects of one capability should be comparable with the preconditions of another.

• Input–output compatibility — outputs should be comparable with downstream input requirements.

• Similarity versus composability — functional resemblance must not be confused with executable composition.

• Composition — atomic capabilities should produce a meaningful composite representation.

• Goal relevance — capabilities should be rankable with respect to desired goal conditions.

• Operational properties — cost, reliability, availability, constraints, and resources must be represented appropriately.

3. Related Representation Strategies

Three conceptual strategies were considered. Generic text embeddings offer strong semantic similarity but do not naturally express directed precondition/effect interfaces. Graph embeddings can encode relationships but can make the formal field semantics less explicit. A structured feature embedding directly follows the assignment's capability tuple and is therefore the selected baseline.

The result is intentionally an assignment-specific design rather than simply applying a pretrained embedding model, which the brief explicitly says is insufficient on its own.

4. Proposed Representation

SCE is organized into seven blocks: identity/mechanism, inputs, outputs, preconditions, effects, resources/constraints, and operational quality.

Block	Representation	Why it matters
Identity	ID, name, type, mechanism	Distinguishes capabilities and implementations
Inputs	Required data specifications	Downstream interface compatibility
Outputs	Produced data specifications	Upstream interface compatibility

Block	Representation	Why it matters
Preconditions	Required state predicates	Applicability
Effects	State changes	Directed composition
Resources / constraints	Feasibility information	Operational validity
Quality	Time, money, resource cost, risk, reliability, availability	Operational comparison

5. Mathematical Formulation

For each structured field F, deterministic token vectors are averaged to produce E(F). The capability embedding is

$$\phi_C(C) = [E_{identity}, E_{input}, E_{output}, E_{precondition}, E_{effect}, E_{resource/constraint}, q]$$

Functional similarity is cosine similarity. Directed compatibility is computed independently from the effect/precondition and output/input boundaries.

$$Comp(C_i,C_j) = 0.6\cos(E_i,P_j) + 0.4\cos(O_i,I_j)$$

6. Capability Composition Model

For a valid sequence $C_1 \rightarrow \ldots \rightarrow C_n$, SCE constructs a composite capability using the same representation. The first capability supplies the external inputs and preconditions; the last supplies the external outputs. Effects, constraints, and resources are combined. Execution time and costs are additive; reliability is multiplicative; availability is the minimum chain availability.

Example. CreateOrder produces OrderExists=true and OrderStatus=CREATED. These satisfy the corresponding requirements of MakePayment, giving the valid composition MakePayment ■ CreateOrder. This is exactly the kind of relationship the assignment asks the embedding to preserve.

7. Implementation

The implementation provides encode_state, encode_goal, encode_capability, compose, similarity, compatibility, and goal_relevance. It uses deterministic structured token vectors and NumPy, avoiding dependence on a pretrained language model.

The implementation deliberately excludes BFS, DFS, A*, D* Lite, LPA*, path search, and replanning because the assignment identifies these as outside scope.

8. Experimental Dataset

The dataset models an e-commerce application and contains ten atomic capabilities spanning API, database, GUI, event, and service mechanisms. It also contains formally specified states, goals, positive and negative compatibility pairs, and operational attributes.

The central compatibility pattern is CreateOrder→MakePayment versus CreateOrder→CancelCart. The first is expected to compose because OrderExists=true is produced by CreateOrder and required by MakePayment. The second is incompatible because CancelCart requires OrderExists=false.

9. Experimental Methodology

• Compatibility experiment — classify labelled directed capability pairs.

• Composition experiment — construct CreateOrder→MakePayment→SendNotification and compare the composite with its atomic components.

• Alternative implementations — compare API, database, and GUI implementations of similar functionality.

• Irrelevant capabilities — rank capabilities by relevance to a specified goal.

• Operational attributes — investigate the effect of cost, reliability, availability, risk, and resources.

10. Results

Compatibility: the supplied experiment achieved 75% accuracy at the chosen 0.60 operating threshold. Because the dataset is small and synthetic, this result is a proof-of-concept rather than a general benchmark.

Composition: the three-step composite CreateOrder→MakePayment→SendNotification achieved a cosine alignment of approximately 0.917 with the aggregate of its atomic vectors. This is encouraging evidence for the intended compositional geometry, but it does not establish compositionality in general.

Alternative implementations: API, database, and GUI variants remain functionally related while remaining distinguishable through mechanism and operational blocks.

11. Analysis

The strongest aspect of the design is the explicit separation between symmetric similarity and directed compatibility. A generic cosine score cannot express the fact that C_i can precede C_j while the reverse order is invalid. The boundary score directly addresses this asymmetry.

A second strength is interpretability. Each region of the vector corresponds to a formal component of the capability model, so an observed relationship can be traced back to interfaces, state predicates, resources, or quality attributes.

The representation also avoids treating alternative mechanisms as identical. CreateOrder, CreateOrderDB, and CreateOrderGUI can have similar functional content while retaining different implementation and quality characteristics.

12. Limitations

• The dataset is synthetic and small.

• The compatibility threshold is an empirical operating point rather than a universal constant.

• Mean token vectors do not fully represent complex logical structure such as nested predicates, quantifiers, or rich inequalities.

• The current composition operator assumes sequential execution and does not model concurrency, branching, rollback, or conditional execution.

• Multiplicative reliability assumes independent capability success probabilities.

• Quality normalization is dataset-dependent.

• The baseline does not learn embedding parameters from a large corpus.

13. Conclusion

SCE provides a complete baseline solution to the assignment's research question. It represents formal application entities in a structured vector space while preserving the distinctions needed for capability compatibility and composition. Most importantly, it does not rely on semantic similarity alone: the representation is paired with a directed compatibility computation that explicitly compares effects to preconditions and outputs to inputs.

The resulting system supports atomic and composite capabilities, goal relevance, alternative implementations, and operational attributes within a single coherent design. The supplied experiments demonstrate feasibility while also making clear that a larger, learned dataset would be required for stronger empirical claims.

14. Deliverables

• Formal embedding design — mathematical specification of the representation, similarity, compatibility, and composition.

• Implementation — Python module implementing the required operations.

• Experimental dataset — states, goals, capabilities, compatibility labels, and operational attributes.

• Technical report — complete methodology, results, analysis, limitations, and conclusion.